



Algoritmo e Programação L02

Lista Alocação dinâmica de memória

Exercícios 01

Ex01: Escreva uma função que receba um byte c (que pode representar um caractere ASCII, por exemplo) e transforme-o em uma string, ou seja, devolva uma string de comprimento 1 tendo c como único elemento

```
//Entrada
#include <stdio.h>
#include <stdlib.h>

char* byte_para_string(char c) {
    // Aloca espaço para uma string de comprimento 1 + 1 (para o caractere nulo)
    char *str = malloc(2 * sizeof(char));
    if (str == NULL) {
        return NULL; // Verifica se a alocação foi bem-sucedida
    }
    str[0] = c; // Coloca o byte c como primeiro elemento
    str[1] = '\0'; // Marca o fim da string com o caractere nulo
    return str;
}

int main(void) {
    char c = 'A';
    char *str = byte_para_string(c);
    if (str != NULL) {
        printf("String resultante: %s\n", str);
        free(str); // Libera a memória alocada para a string
    } else {
        printf("Erro ao alocar memória.\n");
    }
    return 0;
}
```

```
//Saída
String resultante: A
```

Explicação Código:

- **Alocação Dinâmica:** `malloc(2 * sizeof(char))` aloca 2 bytes para a string, onde:
 - O primeiro byte será usado para armazenar o caractere `c`.
 - O segundo byte é o caractere nulo `\0`, que indica o fim da string.
- **Verificação de Alocação:** Verificamos se `malloc` foi bem-sucedido. Se `malloc` retornar `NULL`, isso indica que a alocação falhou, e a função retorna `NULL` também.
- **Preenchimento da String:**
 - `str[0] = c;` armazena o caractere `c` na primeira posição.
 - `str[1] = '\0';` finaliza a string com o caractere nulo.
- **Liberação de Memória:** No `main`, após o

Ex02: Discuta, passo a passo, o efeito do seguinte fragmento de código:

```
//Entrada
int *v;
v = malloc (10 * sizeof (int));
```

Resposta:

- Declaramos um ponteiro `v` do tipo `int *`. Esse ponteiro pode apontar para uma área de memória que armazena inteiros.
- No momento, `v` ainda não aponta para um local específico, então seu valor é indefinido. Em geral, ele não contém um endereço válido.
- A função `malloc` é chamada para alocar memória suficiente para armazenar 10 inteiros. A expressão `10 * sizeof(int)` calcula quantos bytes são necessários:
 - `sizeof(int)` retorna o número de bytes ocupados por um `int` no sistema (normalmente 4 bytes, mas pode variar).
 - Multiplicamos esse valor por 10, pois queremos espaço para 10 inteiros.
 - Se a alocação falhar (por exemplo, se não houver memória suficiente), `malloc` retornará `NULL`, indicando que `v` não aponta para uma memória válida.
- Agora, `v` aponta para um bloco de memória com espaço suficiente para armazenar 10 inteiros consecutivos.

Ex03: Discuta o efeito do seguinte fragmento de código:

```
//Entrada
x = malloc (10 * sizeof *x);
```

Resposta:

- Esse fragmento de código aloca dinamicamente memória suficiente para armazenar 10 elementos do tipo apontado por `x`, utilizando a função `malloc`. Vamos entender o efeito desse código linha por linha.
- O objetivo é alocar espaço para 10 elementos do tipo de `*x`, ou seja, do tipo do valor que `x` aponta.
- `malloc` retorna um `void *`, ou seja, um ponteiro genérico. Este é convertido automaticamente para o tipo apropriado se `x` já for um ponteiro de tipo específico, como `int *`, `float *`, etc.
- Após a alocação, `x` passa a apontar para o bloco de memória alocado, que tem espaço para 10 elementos do tipo de `x`.
- **Generalidade e Flexibilidade:** Usar `sizeof *x` torna o código mais flexível e menos propenso a erros. Caso o tipo de `x` seja alterado (por exemplo, de `int *` para `float *`), não é necessário mudar o código, pois `sizeof *x` se adapta automaticamente ao tipo apontado por `x`.
- **Redução de Erros:** Essa abordagem elimina o risco de calcular erroneamente o tamanho dos elementos caso o tipo de `x` mude futuramente.

Exercícios 02

Ex01: O que há de errado com o seguinte fragmento de código?

```
//Código
int *v;
v = malloc (100 * sizeof (int));
v[0] = 999;
free (v+1);
```

Resposta:

- O problema no código está na linha: `free(v + 1)`
- Em vez de chamar `free(v)`, o código está chamando `free(v + 1)`.
- `v + 1` é o endereço que está um inteiro após o início do bloco de memória alocado. Em outras palavras, ele aponta para o segundo elemento do bloco (equivalente a `&v[1]`).
- Quando você passa um endereço que não é o endereço original retornado por `malloc` para `free`, o comportamento é indefinido. Isso pode causar problemas como vazamentos de memória, corrupção de memória, e até falhas de execução do programa.
- Para liberar o bloco de memória corretamente, sempre passe o endereço inicial retornado por `malloc`, que neste caso é `v`: `free(v)`

Ex02: A seguinte função promete devolver um vetor com os 3 primeiros números primos maiores que 1000. Onde está o erro?

```
//Código
int *primos (void) {
    int v[3];
    v[0] = 1009; v[1] = 1013; v[2] = 1019;
    return v; }
```

Resposta:

- O problema com essa função está no fato de que o vetor `v` é declarado como uma variável local dentro da função `primos`. Variáveis locais são alocadas na **stack** (pilha) e são automaticamente destruídas quando a função termina. Assim, ao retornar o ponteiro `v`, estamos tentando acessar memória que foi desalocada ao sair da função, o que causa **comportamento indefinido**.
- **Vida Útil Limitada do Array `v`:**
 - O array `v` é uma variável local e deixa de existir assim que a função `primos` retorna.
 - O endereço retornado, portanto, passa a apontar para uma área de memória que não é mais válida.
- **Acesso a Memória Inválida:**
 - Se o código chamar `primos` e tentar acessar o vetor retornado, isso poderá levar a resultados imprevisíveis, como acessar dados incorretos ou até mesmo provocar falhas no programa.
- **Solução:** Podemos alocar dinamicamente o vetor usando `malloc` para garantir que ele continue válido após o término da função.

Ex03: A seguinte função promete acrescentar uma célula-cabeça à lista encadeada `lst` e devolver o endereço da nova lista. Onde está o erro?

```
//Código
celula *acrescentaCabeca (celula *lst) {
    celula cabeca;
    cabeca.prox = lst;
    return &cabeca; }
```

```
//Código Corrigido
#include <stdlib.h>

typedef struct celula {
    int dado;
    struct celula *prox;
} celula;
```

Resposta:

- A variável `cabeca` é uma **variável local** dentro da função `acrescentaCabeca`. Assim que a função termina, `cabeca` deixa de existir, e qualquer ponteiro para ela se torna inválido, causando **comportamento indefinido** ao tentar acessar essa memória.
- Vida Útil da Variável Local `cabeca`:**
 - A variável `cabeca` é uma variável local da função, alocada na stack.
 - Quando a função `acrescentaCabeca` retorna, a memória associada a `cabeca` é liberada automaticamente.
 - Retornar `&cabeca` devolve um ponteiro para uma área de memória que não é mais válida assim que a função termina.
- Acesso a Memória Inválida:**
 - Se tentarmos acessar ou manipular o ponteiro retornado (`&cabeca`) fora da função, isso resultará em comportamento indefinido.

```
celula* acrescentaCabeca(celula *lst) {
    celula *cabeca = malloc(sizeof(celula)); // Aloca memória para a célula-cabeça
    if (cabeca == NULL) {
        return NULL; // Verifica se a alocação foi bem-sucedida
    }
    cabeca->prox = lst; // Faz a nova célula-cabeça apontar para a lista existente
    return cabeca; // Retorna o endereço da nova célula-cabeça
}
```

- Alocação Dinâmica:**
 - `cabeca` é agora alocada dinamicamente, então seu espaço de memória continuará válido após o término da função.
 - `malloc` aloca memória na heap, que precisa ser liberada com `free` quando não for mais necessária.
- Configuração do Ponteiro `prox`:**
 - A nova célula `cabeca` aponta para `lst`, tornando-se a nova célula-cabeça da lista.
- Retorno do Ponteiro Válido:**
 - Como `cabeca` é alocada dinamicamente, o ponteiro retornado é válido e permanece acessível após a saída da função.

Ex04: Discuta, passo a passo, o efeito do seguinte fragmento de código:

```
//Código
int *p, *q;
p = malloc (sizeof (int));
*p = 123;
q = malloc (sizeof (int));
*q = *p;
q = p;
free (p);
free (q); // má ideia...
q = NULL; // boa ideia
```

Resposta:

- Declaração de Ponteiros do tipo int:** `int *p, *q;`
- Alocação de Memória para `p`:** `p = malloc(sizeof(int));`
- Atribuição de Valor via `p`:** `*p = 123;`
- Alocação de Memória para `q`:** `q = malloc(sizeof(int));`
- Cópia de Valor para `q`:** `*q = *p;`
- Redirecionamento de `q` para `p`:** `q = p;`
 - Agora, `q` não aponta mais para o bloco de memória alocado na linha `q = malloc(sizeof(int));`.
 - Em vez disso, `q` passa a apontar para o mesmo endereço de `p`, ou seja, `q` e `p` agora compartilham o mesmo bloco de memória.
 - O bloco de memória originalmente alocado para `q` na linha `q = malloc(sizeof(int));` fica inacessível, criando um **vazamento de memória** (não há mais um ponteiro que aponte para ele, então não pode ser liberado).
- Liberação de `p` e `q`:** `free(p);` e `free(q);`
 - `free(p);` libera o bloco de memória apontado por `p`.
 - Depois disso, `q` também aponta para o mesmo bloco liberado, uma vez que `q` foi redirecionado para `p`. Então, `free(q);` tenta liberar a mesma área de memória novamente, o que é **um erro**.
 - Liberar duas vezes a mesma área de memória é um **erro de double-free** e pode causar comportamento indefinido, podendo até corromper o gerenciamento de memória do programa.
- Anulação de `q`:** `q = NULL;`
 - Colocar `q` como `NULL` evita que `q` aponte para uma área de memória inválida, mas essa linha sozinha não resolve o problema principal do código, que é o double-free causado anteriormente.
 - Mesmo que `q` tenha sido anulado aqui, o erro já aconteceu.

Exercícios 03

Ex01: Escreva uma função que desaloque a matriz M alocada acima. Quais devem ser os parâmetros da função?

```
//Código
#include <stdlib.h>

int **M;
M = malloc (m * sizeof (int *));
for (int i = 0; i < m; ++i)
    M[i] = malloc (n * sizeof (int));

// Uso da matriz
// ...

// Desaloca a matriz
desalocaMatriz(M, m);
```

Parâmetros da Função:

A função de desalocação precisa receber:

- O ponteiro para o array de ponteiros `M`.
- O número de linhas `m`, pois é necessário saber quantos subarrays foram alocados para liberar cada um deles.

Portanto, os parâmetros da função serão:

- `int **M`: ponteiro para o array de ponteiros que representam as linhas da matriz.
- `int m`: o número de linhas da matriz.

```
//Função de Desalocação
void desalocaMatriz(int **M, int m) {
    // Libera cada uma das linhas da matriz
    for (int i = 0; i < m; ++i) {
        free(M[i]);
    }
    // Libera o array principal de ponteiros (as linhas)
    free(M);
}
```

Exercícios 04

Ex01: Suponha dado um arquivo de texto que contém uma sequência de números inteiros. O comprimento da sequência é desconhecido. Escreva uma função que imprima esses números em ordem inversa (o último, depois o penúltimo, etc.). É claro que você terá que ler todos os números e armazená-los na memória. A dificuldade está em alocar espaço para uma quantidade de números que só será conhecida quando chegarmos ao fim do arquivo.

Resposta:

```
//Código
#include <stdio.h>
#include <stdlib.h>

void imprime_inverso(const char *nome_arquivo) {
    FILE *arquivo = fopen(nome_arquivo, "r");
    if (!arquivo) {
        perror("Erro ao abrir o arquivo");
        return;
    }

    int *numeros = NULL; // Ponteiro para armazenar os números
    int capacidade = 2; // Capacidade inicial para 2 números
    int tamanho = 0; // Número atual de elementos armazenados
    int numero; //variável temporária é usada para armazenar cada número lido do arquivo,
                //antes de ele ser inserido no array dinâmico numeros

    // Aloca inicialmente um espaço para os números
    numeros = malloc(capacidade * sizeof(int));
    if (!numeros) {
        perror("Erro de alocação de memória");
        fclose(arquivo);
        return;
    }

    // Lê os números do arquivo
    while (fscanf(arquivo, "%d", &numero) == 1) { //&numero == 1 -> indica que é realmente um nro
        // Se o espaço não for suficiente, dobra a capacidade
        if (tamanho >= capacidade) {
            capacidade *= 2;
        }
    }
```

Resposta

Para resolver esse problema, podemos usar a função `realloc` para redimensionar dinamicamente o espaço de memória conforme necessário. A ideia é a seguinte:

- Ler o arquivo uma vez para descobrir o total de números.
- Alocar inicialmente um espaço para um número ou uma pequena quantidade.
- Durante a leitura dos números, aumentar dinamicamente o espaço alocado conforme necessário.
- Após a leitura, imprimir os números em ordem inversa.

Explicação Passo a Passo

- Abrindo o Arquivo:** A função `fopen` abre o arquivo de texto no modo de leitura.
- Alocação Inicial de Memória:** Usa-se `malloc` para alocar inicialmente espaço para dois inteiros.
- Leitura e Redimensionamento com `realloc`:**
 - Lê números com `fscanf`. Se o tamanho do array atingir a capacidade atual, `realloc` dobra o espaço para acomodar mais números.
- Impressão em Ordem Inversa:**
 - Um `for` imprime os números do último ao primeiro, garantindo a ordem inversa.
- Liberação de Memória:** `free` libera a memória alocada dinamicamente para evitar vazamento de memória.

Uso da Função

Para usar a função `imprime_inverso`, chame-a passando o nome do arquivo que contém a sequência de números:

```
//Código
int main() {
    imprime_inverso("numeros.txt");
}
```

```

        int *temp = realloc(numeros, capacidade * sizeof(int));
        if (!temp) {
            perror("Erro ao redimensionar a memória");
            free(numeros);
            fclose(arquivo);
            return;
        }
        numeros = temp; //rearmazena os valores na variavel original
    }
    numeros[tamanho++] = numero; //tamanho é usado 1º e depois incrementado
}

// Imprime os números em ordem inversa
for (int i = tamanho - 1; i >= 0; i--) {
    printf("%d ", numeros[i]);
}
printf("\n");

// Libera a memória alocada
free(numeros);
fclose(arquivo);
}

```

```

    return 0;
}

```

Ex02: Suponha dado um arquivo de texto que contém uma sequência de números inteiros. O comprimento da sequência é desconhecido. Escreva uma função que armazene esses números em um vetor cujo tamanho seja o mínimo necessário. (Leia o arquivo uma só vez!).

Resposta:

```

//Código
#include <stdio.h>
#include <stdlib.h>

int *carrega_numeros(const char *nome_arquivo, int *tamanho_final) {
    FILE *arquivo = fopen(nome_arquivo, "r");
    if (!arquivo) {
        perror("Erro ao abrir o arquivo");
        return NULL;
    }

    int *numeros = NULL;    // Ponteiro para o vetor de números
    int capacidade = 2;    // Capacidade inicial do vetor
    int tamanho = 0;        // Quantidade de números armazenados
    int numero;

    // Aloca inicialmente espaço para 2 números
    numeros = malloc(capacidade * sizeof(int));
    if (!numeros) {
        perror("Erro de alocação de memória");
        fclose(arquivo);
        return NULL;
    }

    // Lê os números do arquivo
    while (fscanf(arquivo, "%d", &numero) == 1) {
        // Se o vetor está cheio, dobra a capacidade
        if (tamanho >= capacidade) {
            capacidade *= 2;
            int *temp = realloc(numeros, capacidade * sizeof(int));
            if (!temp) {
                perror("Erro ao redimensionar a memória");
                free(numeros);
                fclose(arquivo);
                return NULL;
            }
            numeros = temp;
        }
        numeros[tamanho++] = numero;
    }

    // Reduz o vetor para o tamanho exato necessário
    int *resultado = realloc(numeros, tamanho * sizeof(int));
    if (!resultado) {
        perror("Erro ao ajustar o tamanho do vetor");
        free(numeros);
        fclose(arquivo);
        return NULL;
    }

    fclose(arquivo);

    //permitir que a função carrega_numeros comunique o tamanho real do vetor (tamanho)
    //de volta para o código que chamou a função.
    *tamanho_final = tamanho; // Define o tamanho final do vetor

    return resultado;
}

```

Resposta

Para armazenar os números em um vetor do tamanho exato necessário sem ler o arquivo duas vezes, podemos fazer a leitura e alocação de memória de forma incremental. Uma abordagem eficiente para isso é dobrar o tamanho da alocação do vetor cada vez que ele estiver cheio e, ao final, redimensioná-lo com o `realloc` para que tenha o tamanho exato.

Explicação Passo a Passo

- **Abertura do Arquivo:** A função abre o arquivo com `fopen` para leitura.
- **Alocação Inicial:** Alocamos espaço para 2 inteiros, usando uma capacidade inicial baixa.
- **Leitura e Redimensionamento Incremental:**
 - Lemos cada número e verificamos se o vetor está cheio.
 - Quando a capacidade é alcançada, duplicamos o espaço com `realloc`.
- **Redimensionamento Final:** Após ler todos os números, usamos `realloc` para ajustar o vetor ao tamanho exato necessário.
- **Retorno do Resultado:**
 - A função retorna o vetor redimensionado e armazena seu tamanho no parâmetro `tamanho_final`.

Uso da Função

Para usar a função `imprime_inverso`, chame-a passando o nome do arquivo que contém a sequência de números:

```

//Código -> Função que imprime numeros
int main() {
    int tamanho;
    int *numeros = carrega_numeros("numeros.txt", &tamanho);

    if (numeros) {
        printf("Números no vetor:\n");
        for (int i = 0; i < tamanho; i++) {
            printf("%d ", numeros[i]);
        }
        printf("\n");

        free(numeros); // Libera a memória alocada
    }

    return 0;
}

```